

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**
**НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ**
Факультет информационных технологий
Кафедра параллельных вычислений

ОТЧЕТ
О ВЫПОЛНЕНИИ ПРАКТИЧЕСКОЙ РАБОТЫ

«Игра "Жизнь"»

студента 2 курса, 24203 группы

Анисимова Льва Евгеньевича

Направление 09.03.01 – «Информатика и вычислительная техника»

Преподаватель:
доцент кафедры параллельных
вычислений
Андрей Юрьевич Власенко

Новосибирск, 2026

СОДЕРЖАНИЕ

ЦЕЛЬ.....	3
ЗАДАНИЕ.....	3
ОПИСАНИЕ РАБОТЫ.....	4
Приложение 1. <i>Код последовательной программы</i>	6
Приложение 2. <i>Код программы tri.spp</i>	9
Приложение 3. <i>Код скрипта запуска на кластере</i>	14
Приложение 4. <i>Таблицы</i>	16
Приложение 5. <i>Графики</i>	17
Приложение 6. <i>Профилирование</i>	18

ЦЕЛЬ

Практическое освоение методов реализации алгоритмов мелкозернистого параллелизма на крупноблочном параллельном вычислительном устройстве на примере реализации клеточного автомата «Игра "Жизнь" Дж. Конвея» с использованием неблокирующих коммуникации библиотеки MPI.

ЗАДАНИЕ

1. Написать параллельную программу на языке C/C++ с использованием MPI, реализующую клеточный автомат игры "Жизнь" с завершением программы по повтору состояния клеточного массива в случае одномерно декомпозиции массива по строкам и с циклическими границами массива. Проверить корректность исполнения алгоритма на различном числе процессорных ядер и различных размерах клеточного массива, сравнив с результатами, полученными для исходных данных вручную.
2. Измерить время работы программы при использовании различного числа процессорных ядер: 1, 2, 4, 8, 16. Размеры клеточного массива X и Y подобрать таким образом, чтобы решение задачи на одном ядре занимало не менее 30 секунд. Построить графики зависимости времени работы, ускорения и эффективности распараллеливания от числа используемых ядер.
3. Произвести профилирование программы и выполнить ее оптимизацию.
4. Составить отчет.

ОПИСАНИЕ РАБОТЫ

В данной лабораторной работе реализована параллельная версия клеточного автомата «Игра Жизнь» (Дж. Конвея) с использованием стандарта MPI. Размер игрового поля составил 1008x1008 клеток. Игровое поле представляет собой тор (зацикленные границы по горизонтали и вертикали).

Для распараллеливания применялась одномерная доменная декомпозиция: матрица разрезалась на горизонтальные полосы, и каждому из процессов доставалась своя часть строк.

В исходном коде применены следующие особенности:

1. Перекрытие вычислений и обменов: Обмен теньвыми (граничными) строками между соседними процессами выполняется с помощью неблокирующих функций `MPI_Isend` и `MPI_Irecv`. Сразу после запуска обмена процесс начинает вычислять новое состояние внутренних клеток (которые не зависят от соседей). Только после этого вызывается `MPI_Wait`, и процесс дочитывает границы. Это позволяет скрыть сетевые задержки за счет полезных вычислений.

2. Двойная буферизация: Для хранения текущего и следующего поколений используются два массива (`buf0` и `buf1`), указатели на которые меняются местами на каждой итерации (`iter % 2`). Это избавляет от необходимости копировать все поле в памяти.

3. Глобальное условие останова: Программа умеет досрочно завершать работу, если поле переходит в статичное состояние или закликивается (с периодом до 8 шагов). Каждый процесс хранит историю своих состояний (`HIST_WIN = 8`). Если локальный фрагмент совпадает с одним из прошлых, процесс выставляет флаг. Глобальная синхронизация флагов осуществляется через неблокирующую коллективную операцию `MPI_Iallreduce` с побитовым И (`MPI_BXOR`).

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была успешно разработана и протестирована оптимизированная параллельная реализация игры «Жизнь».

Алгоритм корректно отработал условие останова: на сетке 1008x1008 конфигурация стабилизировалась (или вошла в цикл) на итерации 7318, после чего все процессы синхронно завершили вычисления. Это доказывает правильность работы логики `MPI_Allreduce` в связке с локальным окном истории.

Анализ метрик производительности показывает отличную масштабируемость программы:

- На 2 и 4 процессах эффективность близка к идеальной (99% и 97% соответственно).
- При увеличении числа процессов до 24 ускорение составило 20.08, а эффективность сохранилась на высоком уровне — 84%.

Небольшое падение эффективности на большом числе ядер (24 процесса) является естественным и объясняется ростом накладных расходов на пересылку сообщений и коллективную синхронизацию (`MPI_Allreduce`) относительно уменьшившегося объема полезных вычислений в локальной полосе (на 24 процесса каждый узел считает всего по 42 строки). Тем не менее, техника скрытия обменов за неблокирующими вызовами полностью себя оправдала и позволила достичь высоких показателей параллельного ускорения.

Приложение 1. Код последовательной программы

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/time.h>

typedef unsigned char cell_t;

#define CELL(buf, r, c, Y) ((buf)[(r)*(Y)+(c)])
#define HIST_WIN 8

double get_time() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return tv.tv_sec + tv.tv_usec * 1e-6;
}

int main(int argc, char* argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <input_file> [max_iter]\n",
argv[0]);
        return 1;
    }
    int max_iter = (argc >= 3) ? atoi(argv[2]) : 100000;
    int X = 0, Y = 0;

    FILE* f = fopen(argv[1], "r");
    if (!f) { perror("File error"); return 1; }

    char line[100000];
    if (fgets(line, sizeof(line), f)) {
        Y = (int)strlen(line);
        while (Y > 0 && (line[Y-1] == '\n' || line[Y-1] == '\r')) Y--;
        X = 1;
        while (fgets(line, sizeof(line), f)) if (strlen(line) > 1) X+
+;
    }
    rewind(f);

    const int cells = X * Y;
    cell_t* const buf0 = (cell_t*)calloc(cells, sizeof(cell_t));
    cell_t* const buf1 = (cell_t*)calloc(cells, sizeof(cell_t));

    for (int i = 0; i < X; i++) {
        if (fgets(line, sizeof(line), f))
            for (int j = 0; j < Y; j++)
                CELL(buf0, i, j, Y) = (line[j] == '#') ? 1 : 0;
    }
    fclose(f);

    cell_t* hist[HIST_WIN];
    for (int i = 0; i < HIST_WIN; i++)
        hist[i] = (cell_t*)malloc(cells * sizeof(cell_t));
    int hist_count = 0, hist_head = 0;

    memcpy(hist[hist_head], buf0, cells);
    hist_head = (hist_head + 1) % HIST_WIN;
```

```

hist_count = 1;

double t_start = get_time();
int stopped_iter = -1;

for (int iter = 1; iter <= max_iter; iter++) {
    const cell_t* cur = (iter % 2 == 1) ? buf0 : buf1;
    cell_t*      dst = (iter % 2 == 1) ? buf1 : buf0;

    for (int r = 0; r < X; r++) {
        int tr = (r - 1 + X) % X, br = (r + 1) % X;
        for (int c = 0; c < Y; c++) {
            int lc = (c - 1 + Y) % Y, rc = (c + 1) % Y;
            int alive =
                CELL(cur, tr, lc, Y) + CELL(cur, tr, c, Y)
+CELL(cur, tr, rc, Y) +
                CELL(cur, r, lc, Y) +
                CELL(cur, r, rc, Y) +
                CELL(cur, br, lc, Y) + CELL(cur, br, c, Y)
+CELL(cur, br, rc, Y);
            CELL(dst, r, c, Y) = CELL(cur, r, c, Y) ? (alive==2||
alive==3) : (alive==3);
        }
    }

    /* Проверить совпадение dst с историей */
    bool matched = false;
    for (int i = 0; i < hist_count; i++) {
        int idx = (hist_head - 1 - i + HIST_WIN) % HIST_WIN;
        if (memcmp(dst, hist[idx], cells) == 0) {
            stopped_iter = iter;
            matched = true;
            break;
        }
    }

    /* Сохранить снимок dst */
    memcpy(hist[hist_head], dst, cells);
    hist_head = (hist_head + 1) % HIST_WIN;
    if (hist_count < HIST_WIN) hist_count++;

    if (matched) break;
}

double t_total = get_time() - t_start;

printf("t_sequential = %.6f\n", t_total);
printf("X=%d Y=%d\n", X, Y);
if (stopped_iter >= 0)
    printf("stopped_at = %d\n", stopped_iter);
else
    printf("stopped_at = max_iter (%d)\n", max_iter);

int last_iter = (stopped_iter >= 0) ? stopped_iter : max_iter;
const cell_t* final_buf = (last_iter % 2 == 1) ? buf1 : buf0;

FILE* out = fopen("output_seq.txt", "w");
if (out) {

```

```
        for (int i = 0; i < X; i++) {
            for (int j = 0; j < Y; j++)
                fputc(CELL(final_buf, i, j, Y) ? '#' : '.', out);
            fputc('\n', out);
        }
        fclose(out);
        printf("Финальное состояние записано в output_seq.txt\n");
    }

    for (int i = 0; i < HIST_WIN; i++) free(hist[i]);
    free(buf0);
    free(buf1);
    return 0;
}
```


Приложение 2. Код программы *tri.cpp*

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>

typedef unsigned char cell_t;

static int rows_for_rank(int total, int np, int r)
{
    return total / np + (r < total % np ? 1 : 0);
}

static int first_row_for_rank(int total, int np, int r)
{
    int base = total / np, extra = total % np;
    return r * base + (r < extra ? r : extra);
}

#define CELL(buf, r, c, Y) ((buf)[(r)*(Y)+(c)])

/* FNV хэш локальной полосы */
static uint64_t hash_stripe(const cell_t* buf, int local_rows, int Y)
{
    uint64_t h = 14695981039346656037ULL;
    const cell_t* p = buf;
    int n = local_rows * Y;
    for (int i = 0; i < n; i++)
    {
        h ^= (uint64_t)p[i];
        h *= 1099511628211ULL;
    }
    return h;
}

int main(int argc, char* argv[])
{
    MPI_Init(&argc, &argv);

    int rank, np;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &np);
    double t_start = MPI_Wtime();

    if (argc < 2)
    {
        if (rank == 0) fprintf(stderr, "Usage: %s <input_file>
[max_iter]\n", argv[0]);
        MPI_Finalize();
        return 1;
    }

    int X = 0, Y = 0;
    int max_iter = (argc >= 3) ? atoi(argv[2]) : 100000;
    cell_t* global_data = NULL;
```

```

if (rank == 0)
{
    FILE* f = fopen(argv[1], "r");
    if (!f) { X = -1; }
    else
    {
        char line[100000];
        if (fgets(line, sizeof(line), f))
        {
            Y = (int)strlen(line);
            while (Y > 0 && (line[Y - 1] == '\n' || line[Y - 1] ==
'\r')) Y--;
            X = 1;
            while (fgets(line, sizeof(line), f)) if (strlen(line)
> 1) X++;
        }
        rewind(f);
        global_data = (cell_t*)malloc(X * Y * sizeof(cell_t));
        for (int i = 0; i < X; i++)
            if (fgets(line, sizeof(line), f))
                for (int j = 0; j < Y; j++)
                    global_data[i * Y + j] = (line[j] == '#') ?
1 : 0;
        fclose(f);
    }
}

MPI_Bcast(&X, 1, MPI_INT, 0, MPI_COMM_WORLD);
if (X < 0)
{
    if (rank == 0) perror("File error");
    if (global_data) free(global_data);
    MPI_Finalize();
    return 1;
}
MPI_Bcast(&Y, 1, MPI_INT, 0, MPI_COMM_WORLD);

const int local_rows = rows_for_rank(X, np, rank);
const int prev_rank = (rank - 1 + np) % np;
const int next_rank_p = (rank + 1) % np;

    cell_t* const buf0 = (cell_t*)calloc((local_rows + 2) * Y,
sizeof(cell_t));
    cell_t* const buf1 = (cell_t*)calloc((local_rows + 2) * Y,
sizeof(cell_t));

/* Раздача данных */
if (rank == 0)
{
    for (int r = 0; r < np; r++)
    {
        int r_start = first_row_for_rank(X, np, r);
        int r_count = rows_for_rank(X, np, r);
        if (r == 0)
            memcpy(&CELL(buf0, 1, 0, Y), &global_data[0], r_count
* Y);
        else
            MPI_Send(&global_data[r_start * Y], r_count * Y,

```

```

        MPI_UNSIGNED_CHAR, r, 0, MPI_COMM_WORLD);
    }
    free(global_data);
}
else
{
    MPI_Recv(&CELL(buf0, 1, 0, Y), local_rows * Y,
        MPI_UNSIGNED_CHAR, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
}

    int hash_cap = 1024;
    int hash_size = 0;
    uint64_t* hash_hist = NULL;
    if (rank == 0)
    {
        hash_hist = (uint64_t*)malloc(hash_cap * sizeof(uint64_t));
    }

    uint64_t local_h = hash_strip(&CELL(buf0, 1, 0, Y), local_rows,
Y);
    uint64_t global_h = 0;
    MPI_Reduce(&local_h, &global_h, 1, MPI_UINT64_T, MPI_BXOR, 0,
MPI_COMM_WORLD);
    if (rank == 0)
    {
        hash_hist[hash_size++] = global_h;
    }

    int flag_send[1];
    int stopped_iter = -1;

    for (int iter = 1; iter <= max_iter; iter++)
    {
        cell_t* cur = (iter % 2 == 1) ? buf0 : buf1;
        cell_t* dst = (iter % 2 == 1) ? buf1 : buf0;

        /* Обмен граничными строками */
        MPI_Request req[4];
        MPI_Isend(&CELL(cur, 1, 0, Y), Y, MPI_UNSIGNED_CHAR,
            prev_rank, 0, MPI_COMM_WORLD, &req[0]);
        MPI_Isend(&CELL(cur, local_rows, 0, Y), Y, MPI_UNSIGNED_CHAR,
            next_rank_p, 1, MPI_COMM_WORLD, &req[1]);
        MPI_Irecv(&CELL(cur, 0, 0, Y), Y, MPI_UNSIGNED_CHAR,
            prev_rank, 1, MPI_COMM_WORLD, &req[2]);
        MPI_Irecv(&CELL(cur, local_rows+1, 0, Y), Y,
MPI_UNSIGNED_CHAR,
            next_rank_p, 0, MPI_COMM_WORLD, &req[3]);

        /* Внутренние строки */
        for (int r = 2; r <= local_rows - 1; r++)
            for (int c = 0; c < Y; c++)
            {
                int lc = (c - 1 + Y) % Y, rc = (c + 1) % Y;
                int alive =
                    CELL(cur, r-1, lc, Y) + CELL(cur, r-1, c, Y) +
CELL(cur, r-1, rc, Y) +
                    CELL(cur, r, lc, Y) + CELL(cur, r, rc, Y) +

```

```

CELL(cur, r+1, lc, Y) + CELL(cur, r+1, c, Y) +
CELL(cur, r+1, rc, Y);
CELL(dst, r, c, Y) = CELL(cur, r, c, Y) ? (alive == 2
|| alive == 3) : (alive == 3);
}

/* Верхний сосед -> первая строка */
MPI_Wait(&req[0], MPI_STATUS_IGNORE);
MPI_Wait(&req[2], MPI_STATUS_IGNORE);
for (int c = 0; c < Y; c++)
{
    int lc = (c - 1 + Y) % Y, rc = (c + 1) % Y;
    int alive =
        CELL(cur, 0, lc, Y) + CELL(cur, 0, c, Y) + CELL(cur,
0, rc, Y) +
        CELL(cur, 1, lc, Y) + CELL(cur, 1, rc, Y) +
        CELL(cur, 2, lc, Y) + CELL(cur, 2, c, Y) + CELL(cur,
2, rc, Y);
    CELL(dst, 1, c, Y) = CELL(cur, 1, c, Y) ? (alive == 2 ||
alive == 3) : (alive == 3);
}

/* Нижний сосед -> последняя строка */
MPI_Wait(&req[1], MPI_STATUS_IGNORE);
MPI_Wait(&req[3], MPI_STATUS_IGNORE);
{
    int r = local_rows;
    for (int c = 0; c < Y; c++)
    {
        int lc = (c - 1 + Y) % Y, rc = (c + 1) % Y;
        int alive =
            CELL(cur, r-1, lc, Y) + CELL(cur, r-1, c, Y) +
CELL(cur, r-1, rc, Y) +
            CELL(cur, r, lc, Y) + CELL(cur, r, rc, Y) +
            CELL(cur, r+1, lc, Y) + CELL(cur, r+1, c, Y) +
CELL(cur, r+1, rc, Y);
        CELL(dst, r, c, Y) = CELL(cur, r, c, Y) ? (alive == 2
|| alive == 3) : (alive == 3);
    }
}

/* Считаем локальный хэш dst, собираем глобальный через
Iallreduce */
local_h = hash_strip(&CELL(dst, 1, 0, Y), local_rows, Y);
uint64_t global_h_new = 0;
MPI_Request req_stop;
MPI_Iallreduce(&local_h, &global_h_new, 1, MPI_UINT64_T,
MPI_BXOR, MPI_COMM_WORLD, &req_stop);

MPI_Wait(&req_stop, MPI_STATUS_IGNORE);

/* Ранг 0 проверяет совпадение с полной историей */
flag_send[0] = 0;
if (rank == 0)
{
    for (int k = 0; k < hash_size; k++)
    {
        if (hash_hist[k] == global_h_new)

```

```

        {
            flag_send[0] = 1;
            if (rank == 0)
                printf("    [совпадение с итерацией %d,
период=%d]\n",
                    k, iter - k);
            break;
        }
    }
    /* Сохранить хэш */
    if (hash_size == hash_cap)
    {
        hash_cap *= 2;
        hash_hist = (uint64_t*)realloc(hash_hist, hash_cap *
sizeof(uint64_t));
    }
    hash_hist[hash_size++] = global_h_new;
}

/* Раздача флага останова от ранга 0 всем процессам*/
MPI_Bcast(flag_send, 1, MPI_INT, 0, MPI_COMM_WORLD);

if (flag_send[0])
{
    stopped_iter = iter;
    break;
}
}

double t_local = MPI_Wtime() - t_start;
double t_total;
    MPI_Reduce(&t_local, &t_total, 1, MPI_DOUBLE, MPI_MAX, 0,
MPI_COMM_WORLD);

int last_iter = (stopped_iter >= 0) ? stopped_iter : max_iter;
cell_t* final_buf = (last_iter % 2 == 1) ? buf1 : buf0;

int* recvcnts = NULL;
int* displs = NULL;
cell_t* gathered = NULL;
if (rank == 0)
{
    recvcnts = (int*)malloc(np * sizeof(int));
    displs = (int*)malloc(np * sizeof(int));
    for (int r = 0; r < np; r++)
    {
        recvcnts[r] = rows_for_rank(X, np, r) * Y;
        displs[r] = first_row_for_rank(X, np, r) * Y;
    }
    gathered = (cell_t*)malloc(X * Y * sizeof(cell_t));
}

    MPI_Gatherv(&CELL(final_buf, 1, 0, Y), local_rows * Y,
MPI_UNSIGNED_CHAR,
        gathered, recvcnts, displs, MPI_UNSIGNED_CHAR,
        0, MPI_COMM_WORLD);

if (rank == 0)

```

```

{
    printf("t_total      = %.6f\n", t_total);
    printf("X=%d Y=%d\n", X, Y);
    if (stopped_iter >= 0)
        printf("stopped_at = %d\n", stopped_iter);
    else
        printf("stopped_at = max_iter (%d)\n", max_iter);

    FILE* out = fopen("output.txt", "w");
    if (out)
    {
        for (int i = 0; i < X; i++)
        {
            for (int j = 0; j < Y; j++)
                fputc(gathered[i * Y + j] ? '#' : '.', out);
            fputc('\n', out);
        }
        fclose(out);
        printf("Финальное состояние записано в output.txt\n");
    }
    free(recvcounts);
    free(displs);
    free(gathered);
    free(hash_hist);
}

free(buf0);
free(buf1);
MPI_Finalize();
return 0;
}

```

Приложение 3. Код скрипта запуска на кластере

```
#!/bin/bash
#PBS -N GameOfLife_MPI
#PBS -l nodes=2:ppn=12
#PBS -l walltime=03:00:00
#PBS -j oe

cd $PBS_O_WORKDIR

# Параметры задачи
INPUT_FILE="input.txt"
MAX_ITER=1000
# Размеры для генератора (должны соответствовать тому, что ожидает MPI
# версия, если она не умеет автоопределение)
X=2400
Y=2400

echo "=== Подготовка: Генерация поля ==="
# Создаем скрипт генератора на лету или запускаем существующий
python3 -c "
import random
with open('$INPUT_FILE', 'w') as f:
    for _ in range($X):
        row = ''.join(['#' if random.random() < 0.2 else '.' for _ in
range($Y)])
        f.write(row + '\n')
"
echo "Поле $X x $Y сгенерировано в $INPUT_FILE"

echo "=== Компиляция ==="
# Используем -O3 для максимальной производительности
icpc -O3 -std=c++11 -o life_seq ../src/life_seq.cpp
mpicpc -O3 -std=c++11 -o life_mpi ../src/life_mpi.cpp
mpicxx -mpilog -O3 -std=c++11 -o life_mpi_mpe ../src/life_mpi.cpp
echo "Компиляция завершена."

echo ""
echo "===== "
echo " БЛОК 1: Последовательный запуск"
echo "===== "
# ВАЖНО: передаем ИМЯ ФАЙЛА и ИТЕРАЦИИ
./life_seq $INPUT_FILE $MAX_ITER | tee sequential.txt
T_SEQ=$(grep "t_sequential" sequential.txt | awk '{print $3}')
echo "Базовое время T_seq = ${T_SEQ} c"

echo ""
echo "===== "
echo " БЛОК 2: Масштабирование (ускорение и эффективность) "
echo "===== "

NP_LIST="1 2 4 8 12 16 24"

for NP in $NP_LIST; do
    echo ""
    echo "--- NP = ${NP} ---"
    # Здесь убедись, что твоя life_mpi тоже принимает <файл> <итерации>
    # Если MPI версия все еще принимает X Y, замени аргументы на $X $Y
    $MAX_ITER
    if [ "$NP" -le 12 ]; then
        mpirun -machinefile $PBS_NODEFILE -np $NP \
            ./life_mpi $INPUT_FILE $MAX_ITER > result_np${NP}.txt 2>&1
    else

```

```

        mpirun -machinefile $PBS_NODEFILE -np $NP -perhost 12 \
            ./life_mpi $INPUT_FILE $MAX_ITER > result_np${NP}.txt 2>&1
    fi
    cat result_np${NP}.txt
done

echo ""
echo "=====
echo "    Сводная таблица: ускорение и эффективность"
echo "=====
printf "%-4s | %-14s | %-12s | %-11s\n" "NP" "t_total (s)" "Ускорение S"
"Эффект. E"
printf -- "-----|-----|-----|-----\n"
for NP in $NP_LIST; do
    TT=$(grep "t_total" result_np${NP}.txt | awk '{print $3}')
    if [ -n "$TT" ] && [ -n "$T_SEQ" ] && [ "$T_SEQ" != "0" ]; then
        S=$(awk "BEGIN{printf \"%.4f\", ${T_SEQ}/${TT}}")
        E=$(awk "BEGIN{printf \"%.4f\", ${S}/${NP}}")
    else
        S="-"; E="-"
    fi
    printf "%-4s | %-14s | %-12s | %-11s\n" "$NP" "${TT:--}" "$S" "$E"
done

echo ""
echo "=====
echo "    БЛОК 3: Профилирование с MPE (-mpilog), 16 процессов"
echo "=====
mpirun -machinefile $PBS_NODEFILE -np 16 -perhost 12 \
    ./life_mpi_mpe $INPUT_FILE $MAX_ITER > profile_np16.txt 2>&1

TT_P=$(grep "t_total" profile_np16.txt | awk '{print $3}')
if [ -n "$TT_P" ] && [ -n "$T_SEQ" ]; then
    S_P=$(awk "BEGIN{printf \"%.4f\", ${T_SEQ}/${TT_P}}")
    E_P=$(awk "BEGIN{printf \"%.4f\", ${S_P}/16}")
    echo "    Ускорение S(16) = ${S_P},    Эффективность E(16) = ${E_P}"
fi

# Сохранение лога MPE
if [ -f life_mpi_mpe.clog2 ]; then
    mv life_mpi_mpe.clog2 profile_np16.clog2
elif [ -f *.clog2 ]; then
    mv *.clog2 profile_np16.clog2
fi

echo ""
echo "=== Готово ==="
echo "Итоговое поле сохранено в output.txt (от последнего запуска)"
echo "Просмотр профиля: jumpshot profile_np16.clog2"

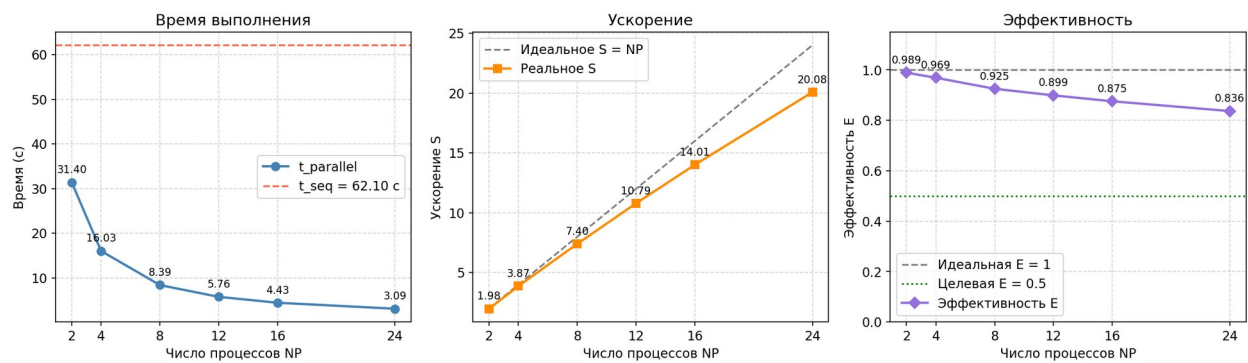
```


Приложение 4. Таблицы

NR	ВРЕМЯ	УСКОРЕНИЕ	ЭФФЕКТИВНОСТЬ	ВРЕМЯ ПОСЛЕДОВАТЕЛЬНОЙ: 62.0958
2	31.4019	1.9775	98.87%	
4	16.0265	3.8746	96.86%	
8	8.3923	7.3992	92.49%	
12	5.7575	10.7853	89.88%	
16	4.4337	14.0055	87.53%	
24	3.0931	20.0756	83.65%	

Приложение 5. Графики

Параллельная игра «Жизнь» на MPI
Сетка 1008×1008, останов на итерации 7318, T_seq = 62.10 c



Приложение 6. Профилирование

